# OR-Select: Deterministic Best-Answer Selection over Stochastic LLM Ensembles via Integer Programming

Aris Dressino[a,*]

[a] *TUM School of Management, Technical University of Munich, Arcisstraße 21, Munich, 80333, Germany*

**Abstract**

Large language models are inherently probabilistic: even at temperature zero, sampling and hardware effects (GPU batch-invariance, floating-point non-associativity) prevent fully reproducible generation. We do not attempt to make generation deterministic. Instead, given a distribution of $N$ candidate answers to the same query, we formulate the selection of a single best answer as a multi-objective integer program over a judge-derived pairwise comparison matrix, with position bias removed via permutation self-consistency. The resulting pipeline is exactly reproducible: the same candidate set and the same judge outputs always yield the same winner, independent of solver internals or execution order. The Copeland-style aggregate we select on is a proven 5-approximation to the intractable exact Kemeny-optimal ranking, so we also report the exact Kemeny-Young solution as a reference baseline wherever ground truth is available. We describe the formal model, a reference implementation using CP-SAT, and an experimental protocol against self-consistency voting and greedy single-sampling baselines across five evaluation tracks (353 ground-truth rows spanning GSM8K, MATH Level 5, AIME 2026, and GPQA Diamond) plus a judge-only validation against RewardBench (161 rows, 90.1% accuracy, a perfect 49/49 on reasoning-category pairs).

*Keywords:* operations research, large language models, integer programming, answer selection, determinism, self-consistency

*Corresponding author. ORCID: 0009-0003-0693-9051.
*Email address:* aris.dressino@tum.de (Aris Dressino)

## 1. Introduction

Large language models are inherently probabilistic: sampling from a token distribution, combined with GPU batch-invariance and floating-point non-associativity, means even greedy (temperature-zero) decoding does not guarantee reproducible output across runs or hardware. A common response is to sample $N$ candidate answers to the same query and pick one – self-consistency [1] does this by majority vote over final answers. Majority vote is well-defined only when candidates cluster: on open-ended queries, where independently sampled answers rarely repeat verbatim, it degenerates to an arbitrary tie-break with no principled basis. We show this empirically in Section 5, not just as a theoretical edge case: on every open-ended query in our experiments, zero of $N = 5$ candidates matched exactly.

We do not attempt to make generation itself deterministic. Instead we treat *selection* among an already-sampled set of $N$ candidates as an optimization problem: an LLM judge produces a pairwise preference matrix over the candidates, and an integer program combines it with a deterministic self-consistency signal, a confidence estimate, and a cost penalty to pick exactly one winner. Given a fixed candidate set, fixed judge outputs, and fixed scores, this selection is provably reproducible – the solve is exact and does not depend on machine, solver internals, or execution order. What is *not* automatically reproducible is whether the judge and scoring calls themselves return the same values on repeated runs, since they are themselves LLM calls; we measure this directly rather than assume it (Section 5).

Our contributions:

1. An integer-programming formulation for best-of-$N$ LLM answer selection that combines an LLM-judged pairwise quality term with a *deterministic*, zero-API-cost self-consistency term, rather than treating self-consistency and LLM-as-judge as competing methods.
2. Two empirical findings about provider-side non-determinism that directly affect any system relying on LLM judges: (a) a provider (seed-supporting, live-tested) returned non-symmetric pairwise scores for byte-identical candidate text, which is mathematically impossible if seeding were honored, and (b) an exact-match short-circuit that closes this specific failure mode entirely by skipping the judge call when it cannot change the mathematically-determined answer.
3. A proof that the tractable Copeland-style aggregate used in selection is a 5-approximation to the exact, NP-hard Kemeny-optimal ranking

(Proposition 3.1), together with an exact Kemeny-Young reference implementation reported alongside it on every ground-truth track, rather than left as an unverified claim.

4. A multi-track evaluation – open-ended queries, a judge-only validation against RewardBench (161 rows, isolating whether the judge itself is accurate), and four ground-truth benchmarks (GSM8K, MATH Level 5, AIME, and GPQA) – because neither divergence nor correctness alone answers whether the method is useful (Section 4).
5. An ablation showing the self-consistency term has had zero effect on any real winner selected so far, and a structural (not coincidental) explanation why, reported as a finding rather than omitted.

## 2. Related Work

Permutation self-consistency [2] establishes that LLM judges exhibit positional bias in listwise ranking and shows that averaging over shuffled orderings toward a Kemeny-style central ranking corrects it; we adopt the pairwise (rather than listwise) form of this technique. PAIRS [3] similarly finds that pairwise comparison plus rank aggregation outperforms asking an LLM evaluator to score or select directly. LLM-PeerReview [4] is the closest prior system: multiple LLM judges score $N$ candidates, aggregated via a Dawid-Skene-style graphical truth-inference model; we aggregate instead via an integer program over a pairwise matrix plus a deterministic self-consistency term. Yang et al. [5] compare Borda, Copeland, and Kemeny-Young aggregation of multiple LLM judges and find Kemeny-Young best aligned with human preference; our Copeland-style $\text{quality}_i$ is a Kemeny-Young approximation, consistent with this result. Position bias as a first-class LLM-judge failure mode is documented by Zheng et al. [6] in the foundational MT-Bench and Chatbot Arena evaluation.

This design sits inside a much older social-choice and rank-aggregation literature that predates LLM judges by decades. Young and Levenglick [7] characterize the Kemeny-Young rule as the unique ranking method that is neutral, consistent, and Condorcet-consistent, the axiomatic justification for treating it as the reference "correct" aggregation rather than an arbitrary choice among many. Bartholdi et al. [8] prove that computing the exact Kemeny-optimal ranking is NP-hard, which is precisely why an exact, transitivity-constrained ILP does not scale as the default choice and

motivates a tractable statistic – our Copeland-style quality$_i$ – with a provable approximation guarantee (Proposition 3.1) instead. Dwork et al. [9] give the broader rank-aggregation lineage this pairwise-matrix approach descends from, originally developed for combining search-engine rankings rather than LLM judgments.

Two recent findings bear directly on this design rather than serving as background. Kohli [10] show that LLM judge panels carry far less independent signal than judge count suggests, since models make correlated errors, and that improved aggregation algorithms close at most 11% of the resulting gap – this bears on whether heterogeneous-provider judging (Section 4) meaningfully reduces correlated error or merely adds cost. Landesberg [11] show that judge calibration and agreement metrics do not predict actual best-of-$N$ selection accuracy, motivating our use of GSM8K [1] as a benchmark with checkable ground truth rather than relying on judge-quality proxies alone.

A third finding explains a result we observed empirically rather than merely predicting it. Kim et al. [12] study 350+ LLMs and find that *stronger* models produce *more* correlated outputs and errors, not less – our GSM8K run (Section 5) found zero candidate disagreement across all 40 problems attempted with a strong generator model, and this is the structural reason why, not an artifact of our specific problem sample. Ding [13] run 265,000 samples on GPQA Diamond and AIME – chosen specifically because they are unsaturated for frontier models, unlike GSM8K – and find agreement is a weak correctness signal (Spearman $\rho = 0.20$-$0.59$) that weakens further for the strongest models. Together these motivate MATH Level 5, AIME, or GPQA Diamond as the benchmark needed to actually test whether this method can correct a wrong majority (Section 6.2), which GSM8K structurally cannot do with a capable generator.

## 3. Method

### *3.1. Pipeline*

Given a query, the pipeline runs five stages (Figure 1).

**Generate.** Sample $N$ candidate answers, either from one model with $N$ distinct seeds or round-robined across multiple providers for heterogeneity. This is the only stage that is not, and is not intended to be, deterministic.

**Score.** Obtain a per-candidate confidence estimate confidence$_i \in [0, 1]$ and a cost proxy cost$_i$ (token count).

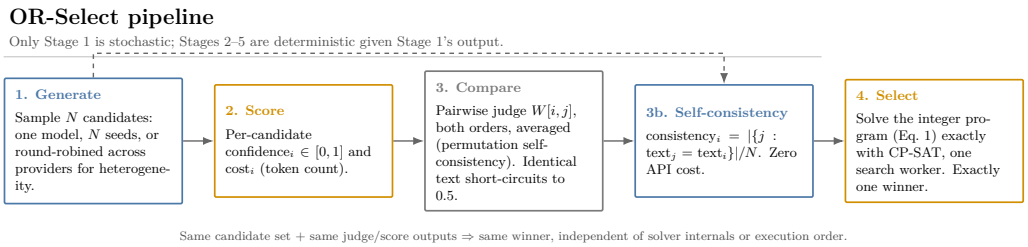



Figure 1: The OR-Select pipeline. Generation (Stage 1) is the only stochastic stage; every later stage is deterministic given Stage 1's output. Stage 3b runs in parallel with Stage 3, from the same candidate texts, at zero additional API cost.

**Compare.** Build a pairwise preference matrix $W[i,j]$: for each unordered pair, an LLM judge is asked twice, once in each text order, and the two votes averaged, following permutation self-consistency [2] – this cancels the positional bias documented in Zheng et al. [6] (Figure 2 illustrates the mechanism). The reverse entry is derived as $W[j,i] = 1 - W[i,j]$ rather than re-judged, since the two are exact complements by construction, halving the judge-call count for a given $N$. Pairs with byte-identical candidate text skip the judge call entirely and receive the mathematically forced $W[i,j] = 0.5$ directly (Section 6.1 motivates why this matters beyond efficiency). Judge-call count therefore scales as $N(N-1)$ in the worst case (2 calls for each of the $\binom{N}{2}$ unordered pairs), the dominant cost of the pipeline: Generate and Score are $O(N)$ in API calls, and Select is a single, small integer program whose solve time is negligible next to the $O(N^2)$ judge calls that produce its input.

**Self-consistency.** Compute $\text{consistency}_i = |\{j : \text{text}_j = \text{text}_i\}|/N$ purely by string comparison – zero additional API calls, and the formal expression of self-consistency voting [1] as one signal rather than the entire decision rule.

**Select.** Solve the integer program below with an exact MIP solver, CP-SAT [14].

### *3.2. Formal Model*

Let $x_i \in \{0,1\}$ indicate whether candidate $i$ is selected.

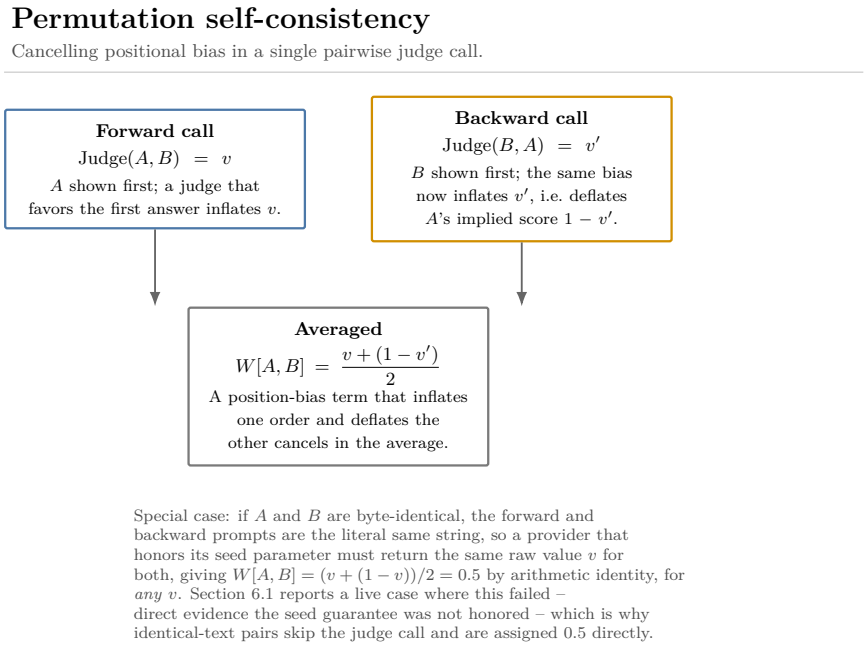



Figure 2: Permutation self-consistency cancels positional bias by averaging the forward and (complement-transformed) backward judge calls for each pair. The special case of byte-identical candidates forces $W = 0.5$ by arithmetic identity alone, independent of the judge's actual output – the basis for both the exact-match short-circuit and the non-determinism finding of Section 6.1.

$$\max \sum_i x_i \left(w_1 \cdot \text{quality}_i + w_2 \cdot \text{confidence}_i + w_4 \cdot \text{consistency}_i - w_3 \cdot \text{cost}_i\right) \tag{1}$$

$$\text{s.t.} \quad \sum_i x_i = 1 \tag{2}$$

$$\text{quality}_i \geq \min_\text{quality} \quad \text{(optional)} \tag{3}$$

$$x_i \in \{0, 1\} \tag{4}$$

$\text{quality}_i = \text{mean}_j W[i, j]$ is the Copeland-style aggregate of the pairwise matrix: the mean weight by which candidate $i$ beats every other candidate. Yang et al. [5] compare Borda, Copeland, and Kemeny-Young aggregation of multiple LLM judges and find Kemeny-Young best aligned with human preference; Copeland is a tractable approximation to it (Section 3.3 makes this precise). Default weights $(w_1, w_2, w_4, w_3) = (1.0, 0.2, 0.3, 0.01)$ were chosen by inspection, not tuned; Section 4.5 reports an ablation on $w_4$.

*Worked example.* Table 1 and Figure 3 decompose Equation (1) term by term for one real Track A run ($N = 5$, no exact-text duplicates). $\text{quality}_i$ is recovered exactly by inverting the saved $\text{score}_i$ against the known $\text{confidence}_i$, $\text{cost}_i$, and $\text{consistency}_i$ for this run – the same inversion used in the self-consistency-term ablation (Section 4.5) – and the recovered ranking's argmax is verified by script to match the run's saved winner before either the table or the figure is generated.

Table 1: Component scores for one real Track A run (winner marked *). $\text{quality}_i$ is recovered by inversion, not directly stored.

| Candidate | $\text{quality}_i$ | $\text{confidence}_i$ | $\text{consistency}_i$ | $\text{cost}_i$ | $\text{score}_i$ |
|---|---|---|---|---|---|
| c0 | 0.606 | 1.00 | 0.20 | 38 | 0.486 |
| c1 | 0.387 | 1.00 | 0.20 | 32 | 0.327 |
| c2* | 0.644 | 1.00 | 0.20 | 33 | 0.574 |
| c3 | 0.575 | 1.00 | 0.20 | 38 | 0.455 |
| c4 | 0.288 | 1.00 | 0.20 | 33 | 0.218 |

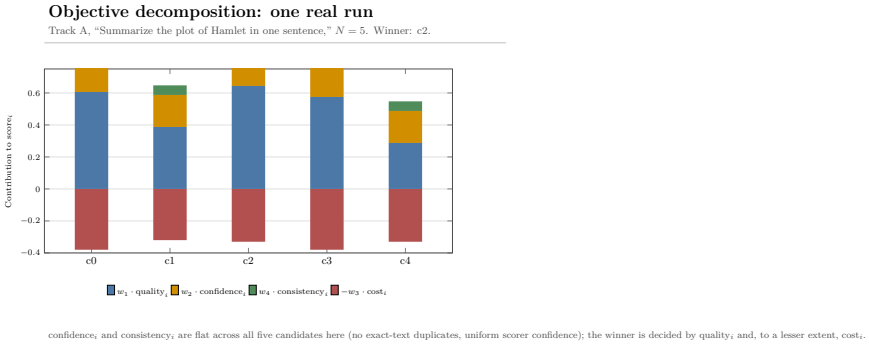



Figure 3: The same run as Table 1, as stacked weighted contributions to $\text{score}_i$. Here $\text{confidence}_i$ and $\text{consistency}_i$ are constant across candidates (no exact-text duplicates; uniform scorer confidence), so the ranking is decided by $\text{quality}_i$ and, to a lesser extent, $\text{cost}_i$ – concretely illustrating why the self-consistency term has not yet been load-bearing on any real winner (Section 4.5).

### *3.3. Exact Kemeny-Young Baseline*

Selecting a single winner by $\text{quality}_i$ does not require solving the full ranking problem, but a full ranking is exactly what the Kemeny-Young rule [7] produces, and computing it exactly gives a reference point against which to measure the Copeland approximation. For distinct candidates $i \neq j$, let $y_{ij} \in \{0, 1\}$ indicate that $i$ is ranked above $j$. The exact Kemeny-Young ranking solves

$$\max \sum_{i \neq j} W[i, j]\, y_{ij} \tag{5}$$

$$\begin{aligned} \text{s.t.} \quad & y_{ij} + y_{ji} = 1 \quad \forall\, i \neq j && (6) \\ & y_{ij} + y_{jk} + y_{ki} \leq 2 \quad \forall \text{ distinct } i, j, k && (7) \\ & y_{ij} \in \{0, 1\} && (8) \end{aligned}$$

Constraint (6) forces a total order on every pair; constraint (7) eliminates 3-cycles, which is necessary and sufficient for the resulting relation to be a valid linear order. The objective (5) maximizes total agreement with the judge's pairwise weights, equivalently (since $\sum_{i \neq j} W[i, j]$ is a fixed constant once $W$ is observed) minimizing total weighted pairwise disagreement – the same quantity Proposition 3.1 bounds. Because $y_{ji} = 1 - y_{ij}$ by (6), the reference implementation (`or_select/kemeny.py`) allocates a variable only

for each unordered pair, giving $\binom{N}{2}$ binary variables and $O(N^3)$ transitivity constraints, one per ordered triple of distinct candidates. Bartholdi et al. [8] prove this problem is NP-hard in general; at the candidate-pool sizes this pipeline targets ($N \leq 6$ in every track reported here), CP-SAT solves it exactly, and we report it alongside quality$_i$'s approximate ranking on every ground-truth track (Section 5) rather than leaving it as future work.

Exact solve time is not uniform in $N$: it depends on how close the pairwise weights are to a coin flip. Measured directly on synthetic instances, a candidate pool with a clear quality gradient solves at $N = 20$ in tens of milliseconds, while near-uniform weights close to 0.5 – the case where the judge is genuinely unsure – can leave the search unfinished after 60 seconds. This does not threaten the determinism guarantee of Section 3.2, since the production selection path (a linear score over $N$ candidates) is a trivial case regardless of $N$; it applies only to the exact Kemeny-Young reference baseline, which is solved within a fixed budget of CP-SAT conflicts rather than wall-clock time, so that "same inputs $\Rightarrow$ same ranking" continues to hold even when the budget is exhausted before optimality is proven – the solver then reports the best ranking found together with an explicit non-optimality flag, rather than silently returning a truncated search as if it were the verified consensus.

### *3.4. Approximation Guarantee*

Ranking candidates by quality$_i$ is exactly the weighted-out-degree ("weighted number of wins") rule studied by Coppersmith et al. [15].

**Proposition 3.1** (15)**.** *Let $W[i,j] + W[j,i] = 1$ for all candidate pairs $i \neq j$ (the probability constraint). Then ranking candidates by non-increasing quality$_i$ = mean$_j$ $W[i,j]$ is a 5-approximation to the exact Kemeny-optimal ranking, measured by total weighted pairwise disagreement – the same objective Section 3.2's transitivity-constrained ILP solves exactly, and whose exact computation Bartholdi et al. [8] show is NP-hard. The bound is tight: an infinite family of tournaments exists for which no tie-breaking rule improves on $5 - \epsilon$.*

The probability constraint holds here by construction (Section 3.1: $W[j,i] := 1 - W[i,j]$, never independently judged), so Proposition 3.1 applies to quality$_i$'s ranking without further assumptions, and the 5-approximation is not a loose artifact of the proof technique that a sharper analysis would remove.

The theorem bounds the *full ranking's* total disagreement with the exact Kemeny consensus; it does not directly bound whether the top-ranked (selected) candidate matches the exact Kemeny winner – a stronger, discrete property the worst-case guarantee does not control on its own. We report the realized top-1 agreement empirically instead (Section 5): across all ground-truth tracks with a Kemeny-Young baseline computed, quality$_i$'s top pick differed from the exact Kemeny-Young winner in 4 of 313 graded problems (the tracks with a Kemeny-Young baseline computed: MATH Level 5, AIME 2026 homogeneous and heterogeneous, and GPQA Diamond homogeneous and heterogeneous – GSM8K has none, Table 3), and in every case where the two disagreed on correctness, quality$_i$'s pick was the correct one (2 of 2, both in GPQA Diamond; the remaining 2 disagreements, both in MATH Level 5, are formatting ties under the same grader limitation discussed in Section 5, not a correctness split). This is evidence that the realized approximation gap is, in practice, far tighter on this pipeline's pairwise weights than the theorem's worst case – not a claim that it must be, since 4 instances is too small a sample to bound a realized ratio, only to observe one directionally.

Given fixed quality$_i$, confidence$_i$, consistency$_i$, and cost$_i$, the solve is exact: CP-SAT with a single search worker returns the same optimum regardless of machine or call order, since the objective is a deterministic linear function of fixed inputs and the feasible region is fixed. This is proved by construction, not measured – what we measure empirically is whether the *inputs* to the solve are themselves stable across runs, since quality$_i$ and confidence$_i$ come from LLM calls (Section 6.1).

## 4. Experiments

### *4.1. Setup*

Unless noted otherwise, all results use OpenRouter (`nvidia/nemotron-3.5-lightning-20260807`) as generator, scorer, and judge. Generation is additionally varied by provider in the heterogeneous variants of Tracks D and E (Section 4.4); scoring and judging use this single model throughout every track, a design choice discussed as a limitation in Section 6.2.

### *4.2. Baselines*

**Self-consistency alone**: majority vote over candidates, with no LLM judge and no optimization. When compared on GSM8K, the vote is taken over each candidate's *extracted final numeric answer*, matching Wang et al.'s

original definition, since two candidates can reach the same correct number through different reasoning text.

**Greedy single-sample**: the first of the $N$ candidates, i.e. no ensembling at all.

### *4.3. Judge validation: RewardBench*

Every downstream track measures the *full pipeline* against baselines; none isolates whether `judge_fn` itself – the pairwise LLM judge quality$_i$ is built from – is accurate. Following Landesberg's caution that judge calibration does not predict selection accuracy, we validate the judge directly, in isolation from the selection pipeline, against RewardBench [16]: for each (prompt, chosen, rejected) triple, we call the identical forward+backward permutation-self-consistency judge code path used in Section 3.1 on exactly those two candidates and check whether the resulting weight favors `chosen`. The sample is $N = 161$, drawn as the first 7 rows of each of RewardBench's 23 subsets (equal per-subset weighting, matching RewardBench's own scoring convention), pre-registered before results were seen.

### *4.4. Evaluation tracks*

**Track A (open-ended, no ground truth)**: three queries with no single correct answer (a plot summary, a policy-strategy question, and a personal-finance question), $N = 5$ candidates each. Measures whether OR-Select's selection *differs* from the baselines and by how much, via score margin.

**Track B (GSM8K, ground truth)**: grade-school arithmetic word problems from the public GSM8K test set [1], each with a single correct numeric answer, $N = 3$ candidates each. Measures whether OR-Select's selection is *correct* more often than the baselines – addressing Landesberg's finding that judge-quality proxies do not predict actual selection accuracy. The final numeric answer is extracted from free text by taking the last number in the response (Appendix – reasoning models frequently restate numbers from the problem statement before giving their final answer, so the first number is often wrong).

**Track C (MATH Level 5, ground truth, unsaturated)**: harder competition-style problems from the Level-5 (hardest) subset of MATH-500 [17], 134 problems fetched via the Hugging Face datasets-server API, $N = 3$ candidates each. Chosen specifically because Kim et al. [12], Ding [13] predict it is not saturated for the model used here, unlike GSM8K – the direct

attempt at testing whether OR-Select can correct a wrong self-consistency majority.

**Track D (AIME 2026)**: recent competition-math problems [18], evaluated with homogeneous generation ($N = 3$, 15/30 valid) and three-vendor heterogeneous generation ($N = 6$, 11/30 valid). The latter varies generators while holding the judge fixed.

**Track E (GPQA Diamond)**: graduate-level biology, chemistry, and physics multiple-choice problems from the 198-question Diamond subset [19], evaluated with homogeneous generation ($N = 3$, 109/198 valid) and the same three-vendor heterogeneous setup ($N = 6$, 71/198 valid). Exact-letter grading avoids the formatting ambiguity of free-form math.

### *4.5. Ablations*

Two components admit direct ablation. The self-consistency weight $w_4$ is ablated fully offline: since the scoring formula is known exactly, $\text{quality}_i$ is recoverable by inversion from each saved run's final score, and re-solving with $w_4 = 0$ isolates the term's effect without any new API calls. Permutation self-consistency is ablated with new judge calls: forward-only judging (no backward-direction averaging) against the same already-generated candidate texts, isolating what bidirectional judging buys over naive single-direction judging.

## 5. Results

### *5.1. Judge validation: RewardBench*

Overall accuracy is 90.1% (mean of category means, $N = 161$, 0 infrastructure failures), in the range typically reported for capable general-purpose LLM judges on this benchmark – the pre-registered threshold for trusting $\text{quality}_i$ as a reasonably calibrated signal. Table 2 breaks this down by category.

The load-bearing number for this paper is Reasoning: a perfect 49/49 across all seven math and code subsets. This directly rules out "the judge is not good enough at math/code" as the explanation for Tracks C-E's near-total agreement below – if disagreement had occurred on those tracks, this judge was capable of using it productively. Safety's weaker score is isolated to one subset (`donotanswer`) traced to a prompt-design gap – the judge instruction rewards "accuracy, completeness, and coherence" with no explicit

Table 2: `judge_fn` accuracy on RewardBench, by category ($N = 161$ total).

| Category | Rows | Accuracy |
|---|---|---|
| Reasoning (math-prm, 6 `hep-*` code subsets) | 49 | 100.0% |
| Chat | 35 | 100.0% |
| Chat Hard (adversarial LLMBar subsets) | 42 | 83.3% |
| Safety | 35 | 77.1% |

safety criterion – not a general weakness in the aggregation method, and orthogonal to every result that follows.

### *5.2. Track A: open-ended queries*

OR-Select disagreed with both baselines on all 3 of 3 queries. Across the 15 candidates generated, zero had byte-identical text to any other candidate for the same query – self-consistency had no true majority to fall back on for *any* query tested, resorting to an arbitrary tie-break every time (which happened to coincide with the greedy pick in all three, since both default to "first candidate" absent other information). Score margins were not uniform: two queries showed a clear quality gap between OR-Select's winner and the runner-up (0.088 and 0.446 respectively, in the pipeline's internal score units), while the third was a near-tie (margin 0.003) between paraphrases of the same balanced answer – reported honestly rather than averaged away, since a paper-level claim of "OR-Select wins" would be misleading applied uniformly to all three.

### *5.3. Track B: GSM8K*

All 40 problems ($N = 3$) were graded. All three methods – OR-Select, self-consistency, greedy – scored 40/40 (100%). This is not a null result so much as the predicted complement to Track A: on every one of the 40 problems, all 3 of 3 candidates independently reached the identical correct numeric answer – zero disagreement across the entire run. A judge cannot differentiate candidates that do not differ, and Kim et al. [12], Ding [13] (Section 2) show this is expected behavior for a strong model on a saturated benchmark, not specific to these 40 problems. MATH Level 5, AIME, and GPQA Diamond are the literature-motivated follow-on tracks reported below.

### *5.4. Track C: MATH Level 5*

We evaluate 107 of the 134 Level-5 problems, $N = 3$ candidates each. All four methods scored 98/107 nominally (91.6%). Nine of the apparent errors were grader false negatives caused by formatting, notation, or ordering that the heuristic grader did not normalize, not genuine wrong answers; no genuinely wrong majority was found.

### *5.5. Track D: AIME 2026*

The homogeneous run evaluates 15 of 30 problems and the heterogeneous run 11 of 30. All four methods scored 100% on valid rows, with zero OR-Select/self-consistency disagreements in either variant.

### *5.6. Track E: GPQA Diamond*

The homogeneous run graded 109/198: OR-Select scored 105/109 (96.3%), self-consistency and greedy each 103/109 (94.5%), and Kemeny-Young 104/109 (95.4%). The heterogeneous run graded 71/198: OR-Select, self-consistency, and greedy each scored 68/71 (95.8%), while Kemeny-Young scored 67/71 (94.4%). The homogeneous run produced 2 OR-Select/self-consistency disagreements, both cases of OR-Select correcting a wrong self-consistency majority (0 in the other direction) – discussed in Section 6.2; the heterogeneous run produced none. Kemeny-Young differed from OR-Select once in each run (2 disagreements total), OR-Select correct both times.

### *5.7. Summary across tracks*

Table 3 and Figure 4 consolidate Tracks B-E. GSM8K (B) and both AIME variants (D, D′) sit at a 100% ceiling for every method – no disagreement was possible. MATH Level 5 (C) is the one track with any grading ambiguity (Section 5, Track C, 9 formatting false negatives out of 107). GPQA Diamond, homogeneous (E) is the only track with a genuine, non-notational OR-Select/self-consistency split, and it favors OR-Select in both cases.

### *5.8. Ablations*

Table 4 summarizes both ablations against the same 3 post-fix Track A candidate sets.

**Self-consistency term**: 0 of 3 winners (the Track A queries) changed when $w_4$ was set to zero. This was not close in any case: the spread between the maximum and minimum consistency$_i$ within a run was exactly 0.000 in

Table 3: Accuracy on graded/valid rows by track and method. '–' marks no Kemeny-Young comparator run.

| Track | Valid/Total | Greedy | Self-cons. | OR-Select | Kemeny-Young |
|---|---|---|---|---|---|
| B — GSM8K | 40/40 | 100.0% | 100.0% | 100.0% | – |
| C — MATH L5 | 107/134 | 91.6% | 91.6% | 91.6% | 91.6% |
| D — AIME, homog. | 15/30 | 100.0% | 100.0% | 100.0% | 100.0% |
| D′ — AIME, heterog. | 11/30 | 100.0% | 100.0% | 100.0% | 100.0% |
| E — GPQA, homog. | 109/109 | 94.5% | 94.5% | **96.3%** | 95.4% |
| E′ — GPQA, heterog. | 71/72 | 95.8% | 95.8% | 95.8% | 94.4% |

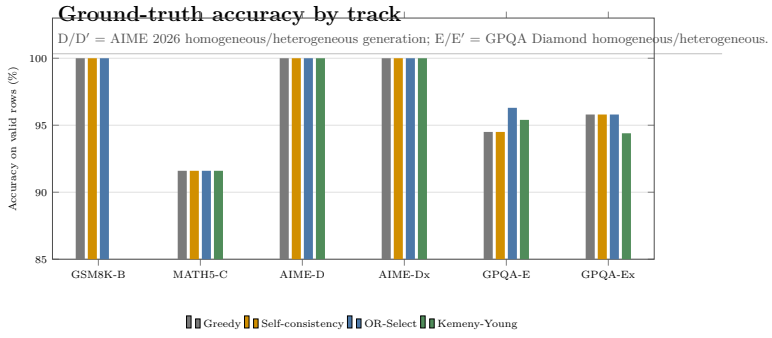



Figure 4: Ground-truth accuracy by track and method, from Table 3. D/D′ = AIME 2026 homogeneous/heterogeneous generation; E/E′ = GPQA Diamond homogeneous/heterogeneous.

Table 4: Ablation results against the 3 Track A candidate sets.

| Component removed | Rent vs. buy | Hamlet | Traffic |
|---|---|---|---|
| Self-consistency term ($w_4 = 0$) | unchanged | unchanged | unchanged |
| Bidirectional judging (forward-only) | **changed** | **changed** | unchanged |

all 3, because every candidate set in these runs was fully diverged (unique text, flat consistency $= 1/N$), and a constant added to every candidate's score cannot change which one is the arg max. The term is designed for the intermediate case, uneven clustering, which no valid live run has yet produced. An earlier pass of this ablation additionally included two single-query smoke tests generated by an earlier pipeline version (predating both the exact-match short-circuit and the self-consistency term itself); their saved scores were not produced by the formula this ablation inverts, and including them silently mixed pre- and post-fix semantics. Both are excluded here; we note the correction rather than the more convenient "0 of 5" headline, since the mistake needed to be caught regardless of whether it changed the conclusion.

**Permutation self-consistency**: forward-only (single-direction) judging changed the winner in 2 of 3 comparisons against the same Track A candidate sets (rent-vs-buy and Hamlet changed; traffic congestion did not). This is the opposite pattern from the self-consistency-term ablation (0 of 3, never): bidirectional judging is doing real, load-bearing work most of the time, consistent with position bias being a first-order LLM-judge failure mode [6, 2] rather than a marginal correction – but 2 of 3 is not "always," and this sample is too small to characterize when the two directions agree versus disagree.

## 6. Discussion

### *6.1. A concrete non-determinism finding*

A live run produced 3 candidates with byte-identical text. By construction, every pairwise weight among identical text must be exactly 0.5: the forward and backward judge calls are the literal same prompt, so $(v+(1-v))/2 = 0.5$ regardless of what the model returns. The observed quality scores were 0.375, 0.50, and 0.625 – direct empirical evidence that the provider's seed parameter is not reliably enforced in practice, not merely a theoretical caveat about vendor documentation. The exact-match short-circuit in Section 3.1 closes this specific failure mode completely, since identical text never reaches the judge at all; it does not address the general case of two *different* texts receiving unstable judge scores across runs, which remains open.

### *6.2. Limitations*

**Scale.** Candidate sets of $N = 3$–$6$ and partial coverage of each benchmark (Table 3) mean this is a preliminary report, not a statistically powered

evaluation. A full-coverage run at larger $N$ is necessary before the accuracy differences in Table 3 can be treated as more than suggestive.

**Single judge model.** Every result uses one model as generator, scorer, and judge simultaneously. The RewardBench validation (Section 5.1) rules out judge *competence* as the explanation for the near-total agreement in Tracks C-E, but says nothing about judge *diversity*: Kohli [10] show that LLM judge panels carry far less independent signal than judge count suggests because models make correlated errors, and that improved aggregation algorithms close at most 11% of the resulting gap. Whether heterogeneous-provider judging would help here, or merely add cost, is untested, not merely under-tested – heterogeneous generation was tested, but heterogeneous judging was not.

**Correcting a wrong majority: rare, and not yet the strongest test of the method.** The most direct test of whether OR-Select adds value over self-consistency is the scenario in which a confident, well-formed majority of candidates converges on a *wrong* answer while a correct minority exists – majority vote fails outright, and only judge-based selection can recover the correct answer. Track E's homogeneous run produced this work's first two cases of OR-Select correcting a self-consistency majority that was wrong, both in OR-Select's favor (Section 5). In neither case did a majority of well-formed candidates converge on a plausible but incorrect answer, however: self-consistency's majority vote degenerated because most candidates failed to produce any parseable answer at all (one garbled, one truncated/empty generation), which OR-Select's judge-based scoring was robust to since it could still identify the one coherent, correct candidate. The only other rows where OR-Select and self-consistency disagree are two Track C rows, both formatting-only variants of the same underlying answer (Section 3.4) rather than a substantive split; across the remaining 349 of 353 valid ground-truth rows they never disagreed at all. The scenario this method is ultimately meant to address – a confident, well-formed wrong majority outvoting a correct minority – remains unobserved in this evaluation, and whether it requires a stronger (more error-prone) generator or simply a larger sample is an open question for future work.

## 7. Conclusion

We formulate best-of-$N$ LLM answer selection as an integer program combining LLM-judged pairwise quality, deterministic self-consistency, con-

fidence, and cost, and prove the tractable Copeland-style aggregate it selects on is a 5-approximation to the exact, NP-hard Kemeny-optimal ranking (Proposition 3.1). The selection step is exactly reproducible given fixed inputs; whether those inputs are themselves stable is an empirical question we measure rather than assume, finding a concrete provider-side nondeterminism failure and closing one instance of it. A judge-only validation against RewardBench (90.1% overall, a perfect 49/49 on reasoning pairs) rules out judge competence as a confound for the tracks that follow. A five-track evaluation shows the method diverges from simpler baselines on open-ended queries, and matches self-consistency on 349 of 353 ground-truth rows tested across GSM8K, MATH Level 5, AIME 2026, and GPQA Diamond – two of the four exceptions are formatting-only ties in MATH Level 5, and the other two are the GPQA Diamond homogeneous run, where OR-Select corrected a self-consistency majority that degenerated on unparseable candidates (Section 6.2). The self-consistency term has not yet influenced a real decision, for a structural reason identified by ablation rather than assumed. This is preliminary, small-scale work; the concrete next steps are paid or frontier-tier evaluation, stronger answer normalization, and a protocol that can reach complete unsaturated benchmark coverage.

## References


[1] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: International Conference on Learning Representations (ICLR), 2023. arXiv:2203.11171, introduces the majority-vote self-consistency baseline this work compares against, and the GSM8K evaluation protocol used here.

[2] R. Tang, X. Zhang, X. Ma, J. Lin, F. Ture, Found in the middle: Permutation self-consistency improves listwise ranking in large language models, in: Findings of the Association for Computational Linguistics: NAACL 2024, 2024. arXiv:2310.07712.

[3] Y. Liu, H. Zhou, Z. Guo, E. Shareghi, I. Vulić, A. Korhonen, N. Collier, Aligning with human judgement: The role of pairwise preference in large language model evaluators, in: Conference on Language Modeling (COLM), 2024. arXiv:2403.16950.

[4] Z. Chen, Z. Ji, Q. Mao, H. Wu, J. Song, J. Cheng, B. Qin, Z. Li, J. Li, K. Sun, Z. Wang, Y. Ban, Z. Sun, X. Ji, H. Sun, X. Huang, Scoring, reasoning, and selecting the best! ensembling large language models via a peer-review process, 2025. arXiv:2512.23213, nearest prior art to OR-Select: aggregates multiple LLM judge scores via a Dawid-Skene-style graphical truth-inference model rather than an integer program over a pairwise matrix.

[5] G. Yang, Y. Liu, S. Miao, X. Liang, Z. Liu, H. Huang, Llms judge themselves: A game-theoretic framework for human-aligned evaluation, 2025. arXiv:2510.15746, compares Borda, Copeland, and Kemeny-Young aggregation of multiple LLM judges; Kemeny-Young (which Copeland, used in this work, approximates) best aligns with human preference.

[6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging llm-as-a-judge with mt-bench and chatbot arena, in: Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2306.05685.

[7] H. P. Young, A. Levenglick, A consistent extension of Condorcet's election principle, SIAM Journal on Applied Mathematics 35 (1978) 285–300. doi:10.1137/0135023, establishes the Kemeny-Young rule as the unique ranking method that is neutral, consistent, and Condorcet-consistent – the social-choice foundation for the exact Kemeny-Young baseline in this work (Section 3.2) and for the approximation guarantee in Proposition 3.1.

[8] J. J. Bartholdi, C. A. Tovey, M. A. Trick, Voting schemes for which it can be difficult to tell who won the election, Social Choice and Welfare 6 (1989) 157–165. doi:10.1007/BF00303169, proves that computing the exact Kemeny-optimal ranking is NP-hard, motivating this work's use of the tractable Copeland-style quality$_i$ statistic (with the Coppersmith et al. approximation guarantee, Proposition 3.1) rather than always solving the exact transitivity-constrained ILP.

[9] C. Dwork, R. Kumar, M. Naor, D. Sivakumar, Rank aggregation methods for the Web, in: Proceedings of the 10th International Conference on World Wide Web (WWW), 2001, pp. 613–622. doi:10.1145/371920.372165, foundational rank-aggregation paper connecting Kemeny-optimal aggregation to Spearman footrule distance and

giving efficient approximation algorithms; the broader OR/social-choice lineage this work's pairwise-matrix aggregation sits in.

[10] G. Kohli, Nine judges, two effective votes: Correlated errors undermine llm evaluation panels, 2026. arXiv:2605.29800, a 9-judge panel carries roughly the independent signal of 2 votes because models make correlated errors; better aggregation math closes at most 11 percent of the gap. Directly relevant to the multi-provider judge configuration (–provider both/all).

[11] E. Landesberg, When llm judge scores look good but best-of-n decisions fail, 2026. arXiv:2603.12520, judge calibration and agreement metrics do not predict actual best-of-N selection accuracy; recommends evaluating on selection/decision accuracy directly, which motivates the GSM8K ground-truth benchmark in this work.

[12] E. Kim, A. Garg, K. Peng, N. Garg, Correlated errors in large language models, in: International Conference on Machine Learning (ICML), 2025. arXiv:2506.07962, 350+ LLMs across two leaderboards and a hiring task: stronger models produce MORE correlated outputs/errors, not less. Structural explanation for zero candidate disagreement observed on GSM8K with a strong generator model in this work.

[13] K. Ding, When llms agree, are they right? auditing self-consistency and cross-model agreement as confidence signals, 2026. arXiv:2607.08065, 265,000 samples on GPQA Diamond and AIME (deliberately chosen as unsaturated benchmarks): agreement is a weak correctness signal (Spearman rho 0.20-0.59), weakest for the strongest models. Motivates MATH Level 5 / AIME / GPQA Diamond as the next benchmark for this work, since GSM8K is saturated for the model used here.

[14] L. Perron, V. Furnon, OR-Tools CP-SAT solver, Google, https://developers.google.com/optimization/, 2023. The exact CP-SAT constraint-programming/MIP solver used as the reference implementation for the integer program in Section 3.2, run single-threaded for reproducibility.

[15] D. Coppersmith, L. Fleischer, A. Rudra, Ordering by weighted number of wins gives a good ranking for weighted tournaments, ACM Transactions on Algorithms 6 (2010) 55:1–55:13. Originally presented at SODA

2006, pp. 776–782. Proves that ranking vertices of a weighted tournament by weighted out-degree (equivalently, weighted number of wins) is a 5-approximation to the exact Kemeny-optimal ranking, provided weights satisfy the probability constraint $w_{uv} + w_{vu} = 1$; also shows the bound is tight (a family of tournaments exists for which no tie-break improves on $5 - \epsilon$). Directly applies to this work's quality$_i$ ranking (Section 3.2), since $W[j, i] := 1 - W[i, j]$ by construction.

[16] N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, N. A. Smith, H. Hajishirzi, Rewardbench: Evaluating reward models for language modeling, in: Findings of the Association for Computational Linguistics: NAACL 2025, 2025. `arXiv:2403.13787`, human-labeled pairwise preference dataset; used to validate the judge_fn component in isolation (49/49 on Reasoning-category pairs), independent of the full selection pipeline.

[17] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, K. Cobbe, Let's verify step by step, 2023. `arXiv:2305.20050`, introduces the process-supervision-trained PRM and, as a byproduct, MATH-500 – a 500-problem curated subset of the Hendrycks MATH test set, later packaged as HuggingFaceH4/MATH-500. This work uses only its Level-5 (hardest) subset, 134 problems, as Track C – MATH-500 is unsaturated for the model used here, unlike GSM8K, so it is the direct test of hypothesis sub-claim 5.

[18] M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, M. Vechev, MathArena: Evaluating LLMs on uncontaminated math competitions, SRI Lab, ETH Zurich, `https://matharena.ai/`, 2025. Source of the AIME 2026 (Track D) problem set (`MathArena/aime_2026`, 30 problems from the two 2026 AIME sittings), curated specifically to be released after model training cutoffs to resist contamination.

[19] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, S. R. Bowman, Gpqa: A graduate-level google-proof q&a benchmark, in: Conference on Language Modeling (COLM), 2024. `arXiv:2311.12022`, introduces the 198-question Diamond subset used as Track E, a graduate-level multiple-choice benchmark designed to resist casual search and non-expert answering.